

Database Systems — Lab Work

Relational Model & Keys — Solutions

Name: Mergaliyev Yersultan

Part 1: Key Identification Exercises

Task 1.1 — Superkey and Candidate Key Analysis

Relation: Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

1. At least 6 superkeys

- {EmpID}
- {SSN}
- {Email}
- {EmpID, SSN}
- {EmpID, Name}
- {SSN, Department}
- {Email, Phone}
- {EmpID, SSN, Email, Phone, Name, Department, Salary} (the whole relation)

Any superset of a candidate key is a superkey — that is why the last three sets above still qualify even though they carry redundant attributes.

2. Candidate keys

A candidate key must be a minimal superkey (no attribute can be removed without losing uniqueness):

- {EmpID} — system-assigned, unique per employee
- {SSN} — unique per person
- {Email} — unique per employee (assuming the company enforces one address per person)

Phone, Name and Department are NOT candidate keys: phone numbers can be shared (e.g. a department landline), names can repeat, and Department clearly repeats across employees.

3. Choice of primary key

EmpID is the best choice for primary key. Reasons:

- It is short, numeric, and efficient for indexing and joins.
- It never changes over an employee's career, unlike Email (can change) or Phone.
- Unlike SSN, it avoids exposing sensitive government-issued identifiers in every foreign key reference and application log.
- It is guaranteed unique by the organization itself rather than by an outside authority.

4. Can two employees share the same phone number?

Yes, it is possible. Nothing in the schema declares Phone as a key or UNIQUE, and in practice employees can share an office extension or a department line. The sample data happens to show three different numbers, but a sample of three rows cannot prove a uniqueness constraint — it only shows that no duplicate occurred in this particular snapshot. Unless the business rules explicitly say "each employee has a private, unique phone number," Phone should not be treated as unique.

Relation B: Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

1. Minimum attributes for the primary key

Primary key: {StudentID, CourseCode, Section, Semester, Year}

2. Why each attribute is necessary

- StudentID — distinguishes which student the registration record belongs to.
- CourseCode — distinguishes which course was taken.
- Section — required because the rule says a student cannot register for the SAME section in the same semester, implying different sections of the same course in the same semester are distinct records.
- Semester + Year — required because the rule explicitly allows the same student to take the same course again in a different semester/year, so semester and year must be part of the key to keep those rows distinct.

Grade and Credits are not part of the key — they are descriptive facts that depend on the whole key (Credits actually depends only on CourseCode+Section+Semester+Year, which is a partial dependency worth flagging if this table were normalized further).

3. Additional candidate keys

Additional candidate keys: None — no other combination of the given attributes is guaranteed unique by the business rules.

Task 1.2 — Foreign Key Design

Table.Column (FK)	References	Meaning
Student.AdvisorID	Professor.ProfID	Each student has one academic advisor, who is a professor
Course.DepartmentCode	Department.DeptCode	Each course belongs to one department
Department.ChairID	Professor.ProfID	Each department's chair is a professor
Enrollment.StudentID	Student.StudentID	Each enrollment record belongs to one student
Enrollment.CourseID	Course.CourseID	Each enrollment record is for one course

Part 2: ER Diagram Construction

The written analysis below (entities, attributes, relationships, cardinalities) should be transferred into an actual diagram using ERDPlus, as required by the lab tools list. Draw rectangles for entities, ellipses for attributes (double ellipse for multi-valued, dashed for derived), diamonds for relationships, and double rectangles/diamonds for weak entities and their identifying relationships.

Task 2.1 — Hospital Management System

Entities

Entity	Type	Notes
Patient	Strong	PK: PatientID
Doctor	Strong	PK: DoctorID
Department	Strong	PK: DeptCode
Appointment	Strong (associative)	PK: AppointmentID; links Patient and Doctor
Prescription	Weak	Partial key: PrescriptionLineNo; owner: Appointment
Room	Weak	Partial key: RoomNumber; owner: Department (room numbers repeat across departments)

Attributes

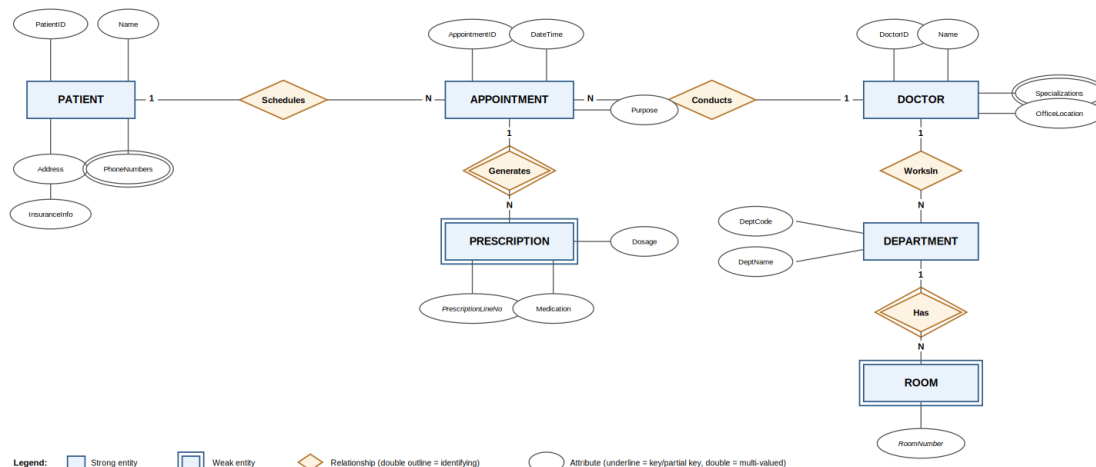
- Patient: PatientID (simple, key), Name (composite: FirstName, LastName), BirthDate (simple), Age (derived, from BirthDate), Address (composite: Street, City, State, Zip), PhoneNumbers (multi-valued), InsuranceInfo (composite: Provider, PolicyNumber).
- Doctor: DoctorID (simple, key), Name (composite), Specializations (multi-valued), PhoneNumber (simple), OfficeLocation (simple).

- Department: DeptCode (simple, key), Name (simple), Location (simple).
- Appointment: AppointmentID (simple, key), DateTime (composite: Date, Time), Purpose (simple), Notes (simple).
- Prescription: PrescriptionLineNo (partial key), Medication (simple), Dosage (simple), Instructions (simple).
- Room: RoomNumber (partial key) — no other attributes given.

Relationships and cardinalities

- Patient — Appointment — Doctor: a patient can have many appointments and a doctor can have many appointments, so this is modeled as an M:N relationship materialized through the Appointment entity (each Appointment row has exactly one PatientID and one DoctorID).
- Doctor — WorksIn — Department: N:1 (each doctor belongs to one department; a department has many doctors).
- Doctor — Prescribes — Prescription — For — Patient: prescriptions are issued during an appointment, so Prescription is a weak entity identified by (AppointmentID, PrescriptionLineNo).
- Department — Has — Room: 1:N identifying relationship (a room cannot exist without its department; PK of Room becomes (DeptCode, RoomNumber)).

ER Diagram



Task 2.2 — E-commerce Platform

Entities

Entity	Type	Notes
Customer	Strong	PK: CustomerID
Product	Strong	PK: ProductID
Category	Strong	PK: CategoryID
Vendor	Strong	PK: VendorID
Order	Strong	PK: OrderID
OrderItem	Weak	Partial key: LineNumber; owner: Order — an order item cannot exist without its order
Review	Strong (associative)	PK: ReviewID; links Customer and Product
Inventory	Strong (or 1:1 extension of Product)	PK: ProductID
Address	Strong	PK: AddressID; type attribute distinguishes billing vs. shipping

Weak entity justification

OrderItem is a weak entity: a line item (product, quantity, price-at-time-of-order) has no independent meaning or identity outside the Order it belongs to. Its key is the owner's key plus a partial key: (OrderID, LineNumber).

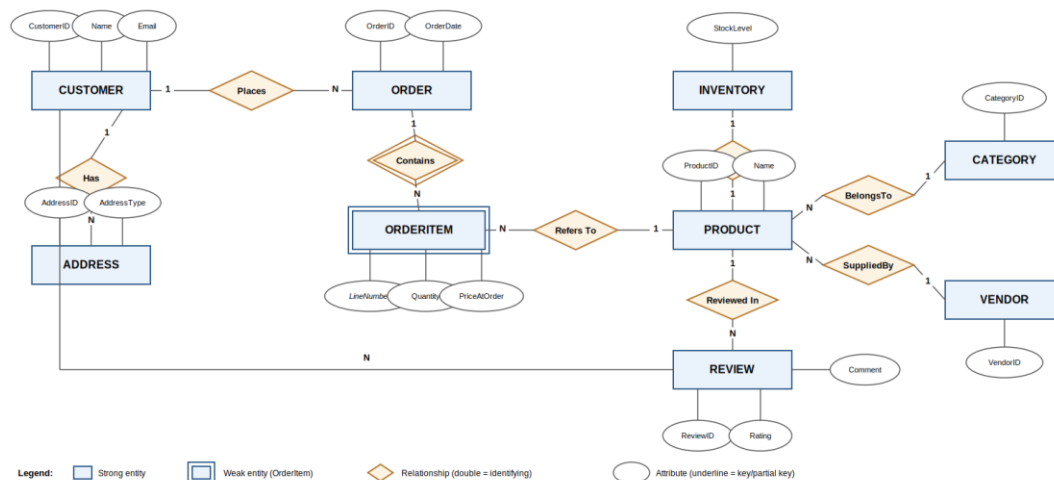
Many-to-many relationship needing its own attributes

Order — contains — Product is an M:N relationship (an order has many products, a product appears in many orders) and it needs its own attributes — Quantity and PriceAtOrderTime — which is exactly why it is materialized as the OrderItem entity rather than a plain relationship line.

Other relationships

- Customer — places — Order: 1:N
- Product — belongsTo — Category: N:1
- Product — suppliedBy — Vendor: N:1 (or M:N if a product can have multiple vendors)
- Customer — writes — Review — about — Product: M:N via the Review entity (attributes: Rating, Comment, Date)
- Product — has — Inventory: 1:1 (stock level, warehouse location)
- Customer — has — Address: 1:N, with an AddressType attribute (billing / shipping) so a shipping address can differ from the billing address

ER Diagram



Part 4: Normalization Workshop

Task 4.1 — Denormalized Table Analysis

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

1. Functional dependencies

- StudentID → StudentName, StudentMajor
- ProjectID → ProjectTitle, ProjectType, SupervisorID

- SupervisorID → SupervisorName, SupervisorDept
- StudentID, ProjectID → Role, HoursWorked, StartDate, EndDate

2. Problems

Redundancy: StudentName/StudentMajor repeat once per project that student is on;
ProjectTitle/ProjectType/Supervisor* repeat once per student assigned to that project;
SupervisorName/SupervisorDept repeat once per project that supervisor runs.

- Update anomaly: if a student changes major, every row for that student must be updated — missing one leaves inconsistent data.
- Insert anomaly: a new project with no students yet cannot be recorded, because the key requires both StudentID and ProjectID.
- Delete anomaly: removing the last student from a project deletes the only rows that stored the project's title, type and supervisor — that information is lost.

3. 1NF

No 1NF violations are visible: every attribute listed holds a single atomic value (assuming, e.g., a student has exactly one major and one role per project). If in practice a student could hold multiple roles on the same project simultaneously, Role would need to be split out — but as given, the table is already in 1NF.

4. 2NF

Primary key: (StudentID, ProjectID).

Partial dependencies (attributes depending on only part of the key):

- StudentID → StudentName, StudentMajor (depends only on StudentID)
- ProjectID → ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept (depends only on ProjectID)

2NF decomposition:

- Student(StudentID, StudentName, StudentMajor) — PK StudentID
- Project(ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept) — PK ProjectID
- Assignment(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate) — PK (StudentID, ProjectID), FKs to Student and Project

5. 3NF

Transitive dependency inside Project: ProjectID → SupervisorID → SupervisorName, SupervisorDept.

Final 3NF schema:

- Student(StudentID, StudentName, StudentMajor) — PK StudentID
- Supervisor(SupervisorID, SupervisorName, SupervisorDept) — PK SupervisorID
- Project(ProjectID, ProjectTitle, ProjectType, SupervisorID) — PK ProjectID, FK SupervisorID → Supervisor
- Assignment(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate) — PK (StudentID, ProjectID), FK StudentID → Student, FK ProjectID → Project

Task 4.2 — Advanced Normalization (BCNF)

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

1. Primary key (the tricky part)

PK = (StudentID, TimeSlot, Room). It is NOT (StudentID, CourseID): a room+time slot pair already pins down exactly one section (one course, one instructor, per the business rules), so (TimeSlot, Room) identifies the section, and adding StudentID identifies which student sits in that section. Using CourseID instead of TimeSlot+Room would fail if the same course had two different sections running at once.

2. Functional dependencies

- $\text{StudentID} \rightarrow \text{StudentMajor}$
- $\text{CourseID} \rightarrow \text{CourseName}$
- $\text{InstructorID} \rightarrow \text{InstructorName}$
- $\text{Room} \rightarrow \text{Building}$ (rooms are unique campus-wide, so the room alone fixes the building)
- $\text{TimeSlot}, \text{Room} \rightarrow \text{CourseID}, \text{InstructorID}$ (only one section can occupy a given room at a given time)
- $\text{InstructorID}, \text{TimeSlot} \rightarrow \text{Room}, \text{CourseID}$ (an instructor can only be in one place at a given time)

3. Is it in BCNF?

No. For a relation to be in BCNF, every determinant (left side of a nontrivial FD) must be a superkey. Here StudentID , CourseID , InstructorID and Room are all determinants but none of them alone is a superkey (the actual key needs three attributes) — so the table violates BCNF several times over.

4. BCNF decomposition

Repeatedly split off each violating FD:

- $\text{StudentMajorTbl}(\text{StudentID}, \text{StudentMajor})$ — PK StudentID
- $\text{CourseInfo}(\text{CourseID}, \text{CourseName})$ — PK CourseID
- $\text{InstructorInfo}(\text{InstructorID}, \text{InstructorName})$ — PK InstructorID
- $\text{RoomInfo}(\text{Room}, \text{Building})$ — PK Room
- $\text{Section}(\text{TimeSlot}, \text{Room}, \text{CourseID}, \text{InstructorID})$ — PK $(\text{TimeSlot}, \text{Room})$; note $(\text{InstructorID}, \text{TimeSlot})$ is an alternate candidate key of this table, and both determinants are now keys, so Section is in BCNF
- $\text{Enrollment}(\text{StudentID}, \text{TimeSlot}, \text{Room})$ — PK $(\text{StudentID}, \text{TimeSlot}, \text{Room})$; FK $(\text{TimeSlot}, \text{Room}) \rightarrow \text{Section}$, FK $\text{StudentID} \rightarrow \text{StudentMajorTbl}$

5. Loss of information

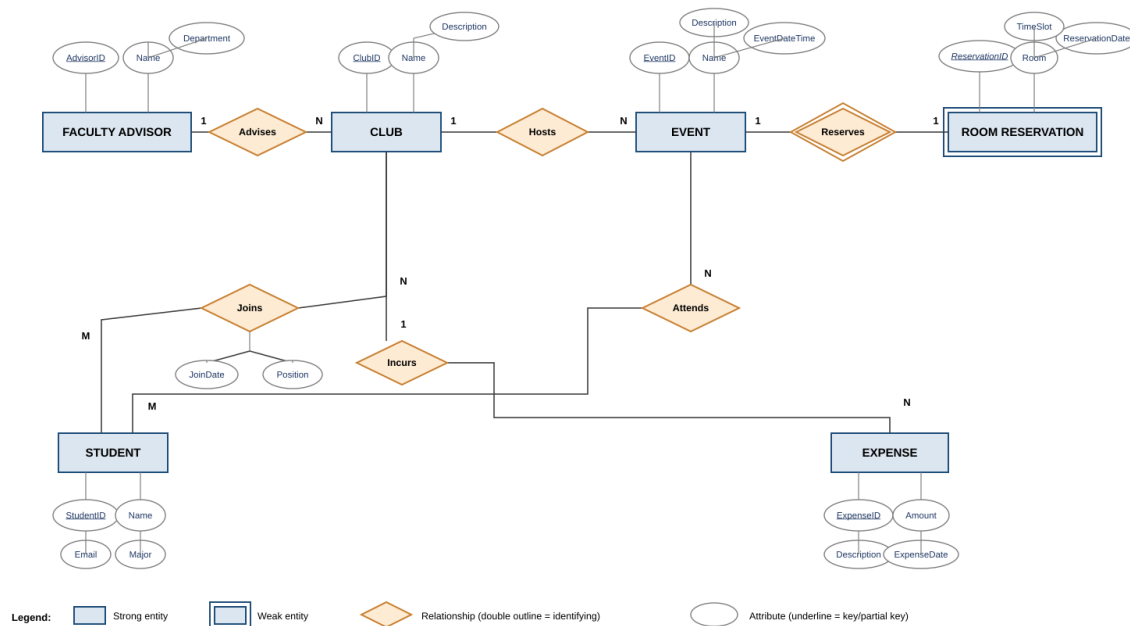
None. This is a standard lossless-join BCNF decomposition: at every step we split off a table on a determinant that becomes its primary key, so joining Section back with Enrollment on $(\text{TimeSlot}, \text{Room})$ reconstructs the original rows exactly, with no spurious tuples. The only thing that is technically lost is dependency preservation for the rule "an instructor teaches only one section at a time" — that constraint still holds in the data but can no longer be enforced by a simple key constraint on a single table; it would need to be checked with a trigger or application logic.

Part 5: Design Challenge — Student Clubs System

1–2. ER diagram (description) and normalized relational schema

Table	Columns	Notes
Student	StudentID (PK), Name, Email, Major	
FacultyAdvisor	AdvisorID (PK), Name, Department	Faculty can advise multiple clubs
Club	ClubID (PK), Name, Description, AdvisorID (FK)	Each club has exactly one advisor (N:1 $\text{Club} \rightarrow \text{Advisor}$)
Membership	StudentID (FK), ClubID (FK), JoinDate, Position	PK $(\text{StudentID}, \text{ClubID})$; resolves M:N $\text{Student} \leftrightarrow \text{Club}$; Position defaults to 'member'
Event	EventID (PK), ClubID (FK), Name, EventDateTime, Description	1:N $\text{Club} \rightarrow \text{Event}$
Attendance	EventID (FK), StudentID (FK)	PK $(\text{EventID}, \text{StudentID})$; resolves M:N $\text{Event} \leftrightarrow \text{Student}$
RoomReservation	ReservationID (PK), EventID (FK), Room, ReservationDate, TimeSlot	1:1 or 1:N $\text{Event} \rightarrow \text{Reservation}$ depending on multi-room events
Expense	ExpenseID (PK), ClubID (FK), Amount, Description, ExpenseDate	Tracks club budget spending over time

ER Diagram



3. Design decision with multiple valid options

Modeling club officer positions was a genuine design choice:

- Option A (chosen): store Position as an attribute directly on Membership. Simple, and enough if the club only needs to know who currently holds which position.
- Option B (rejected): a separate OfficerHistory entity (StudentID, ClubID, Position, StartDate, EndDate) that keeps a full historical record of every position a student has ever held.

Option A was chosen for simplicity and because the requirements only ask to track current officer positions, not historical ones; Option B would be the better choice if the university later wants leadership-history reports.

4. Example queries (in English, not SQL)

- Find all students who are officers in the Computer Science Club.
- List all events scheduled for next week together with their room reservations.
- List the total expenses recorded for each club this semester, ordered from highest to lowest spender.